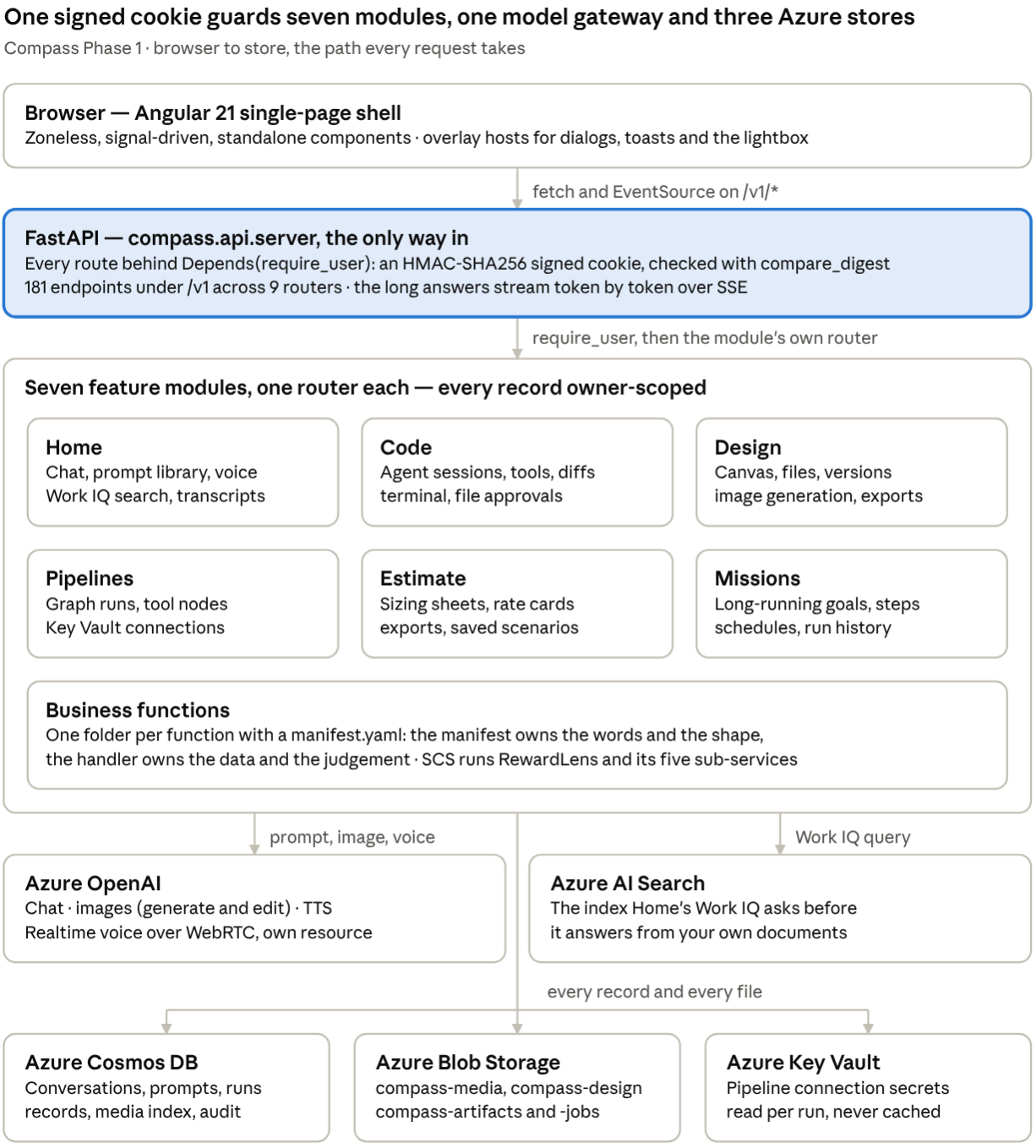
Compass — Architecture & Feature Reference

Oct 3, 2026 · @Manish Kumar

Compass is an agentic workspace: one Angular 21 front end over a FastAPI backend, seven modules sharing one identity, one model gateway and one storage layer. Phase 1 ships 181 HTTP endpoints across ~69,400 lines of Python and ~53,300 lines of Angular front end, with every record in Azure Cosmos DB and every byte in Azure Blob Storage.

At a glance: the whole system

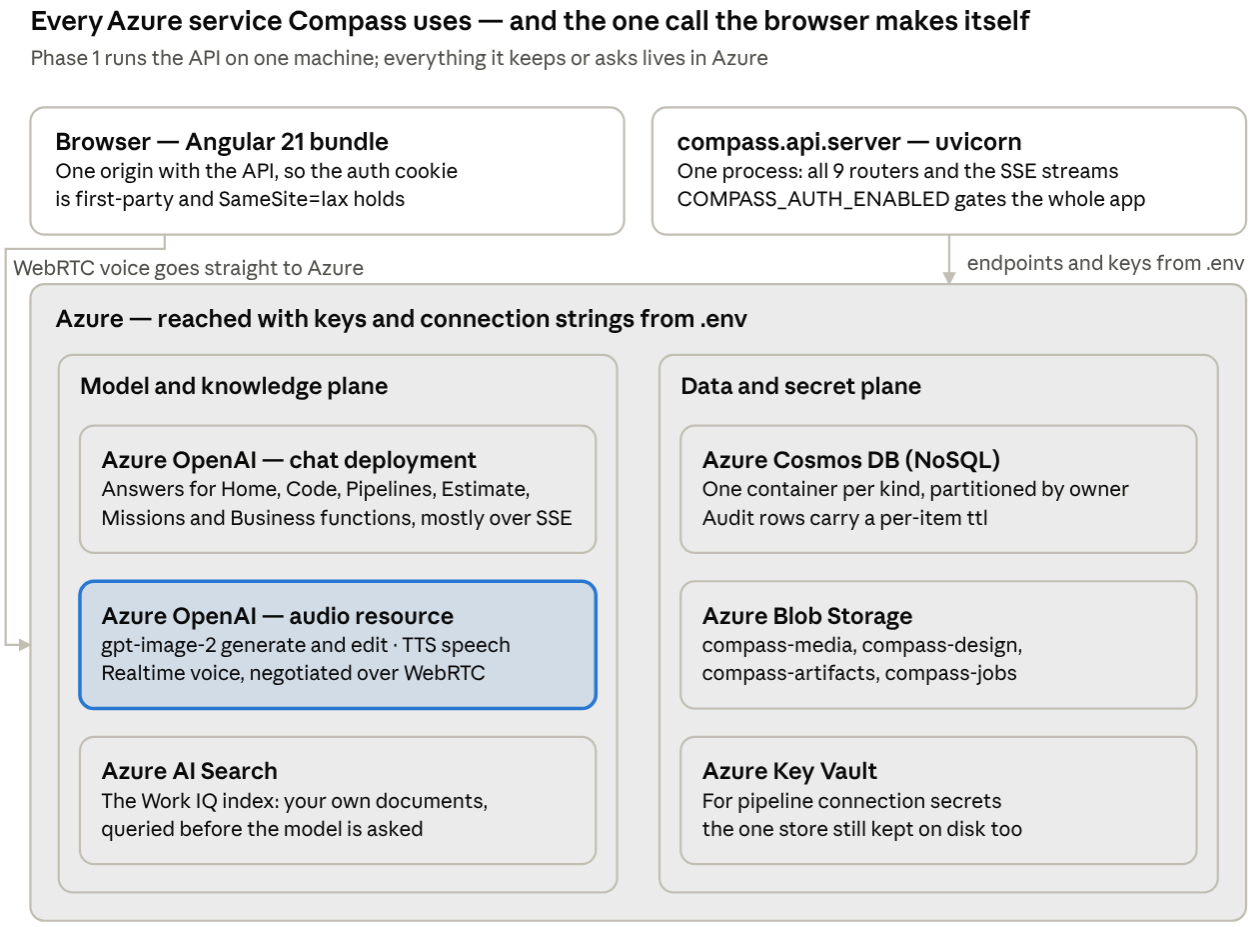


whole system · browser, API gate, seven modules, gateway, three stores

Read it downward. The browser is one Angular application; every call it makes enters FastAPI at `/v1/*`, and the accented band is the only door — no route in Compass is reachable without the signed cookie. Below the door sit the modules — six when this was drawn, seven since Business functions joined them — each a router of its own, each scoping every record it reads to the person who owns it. The two boxes under them are

the only services Compass asks questions of; the three at the foot are the only places it keeps anything. Nothing in Phase 1 persists to the local disk except the pipeline secrets file, which is why Key Vault appears at the bottom right.

The Azure infrastructure



Azure footprint · two runtime pieces, six services, one direct path

Six Azure services, two planes. Everything in the left plane is something Compass **asks**; everything in the right plane is something it **keeps**. The server holds every credential — they are read from `.env` at start-up and never reach the browser.

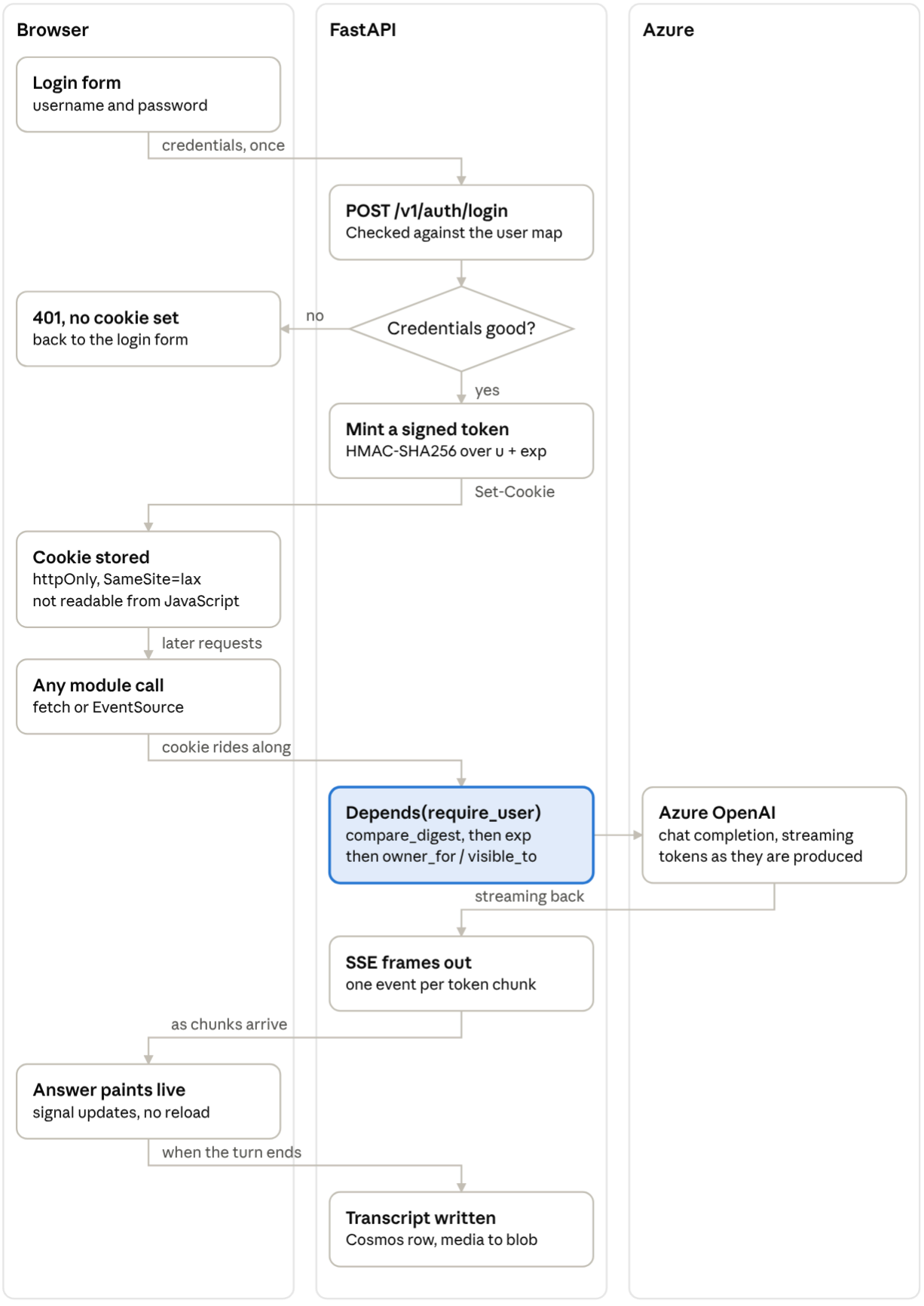
The highlighted box is the single exception to that rule, and it is worth understanding. Realtime voice cannot be proxied: the browser has to open a WebRTC peer connection to the Azure OpenAI audio resource itself. The server still holds the real key; it mints a short-lived ephemeral credential, hands only that to the page, and the page negotiates directly. Note also that image generation and speech live on the *audio* resource rather than the chat one — `gpt-image-2` is deployed there, which is a deployment fact worth knowing before anyone tries to point it at the chat endpoint.

Key Vault is the one line in this picture that is not yet finished. It is wired for pipeline connection secrets, but those secrets are still also written to `data/secrets/connections.json` on the machine running the API. Closing that gap needs a vault URL, and it is the last item standing between Phase 1 and a server that keeps nothing locally.

Sign-in and the request path

The cookie is minted once and re-checked on every call that follows

Sign-in, then one streamed answer, end to end



sign-in to streamed answer · 3 lanes, 1 decision

The top third happens once a session; everything below `Cookie stored` happens again on every single call, and that repetition is the point of the picture.

The token itself is `base64(payload).base64(signature)`, where the payload is `{"u": username, "exp": …}` and the signature is HMAC-SHA256 over it using the server's secret. Nothing in the cookie is encrypted — anyone holding it can read the username — but nothing in it can be *changed*, because `require_user` recomputes the signature with `hmac.compare_digest` and rejects a mismatch in constant time. A tampered or expired cookie takes the same road as a wrong password: 401, and the front end routes back to the login form.

The highlighted step is where authentication becomes authorisation. `require_user` returns a username; the module router then narrows every query to that person through `owner_for`, `visible_to` and `owned`. Those three live in one file on purpose — scoping is the kind of rule that rots when it is copied into seven modules, and a single implementation is the only way a reviewer can confirm that Home, Code, Design, Pipelines, Estimate, Missions and Business functions all enforce it the same way. They deliberately fail *open* on an empty owner, which is what keeps a single-user deployment with `COMPASS_AUTH_ENABLED=0` working without a parallel code path.

The last step is easy to overlook and matters for reading the rest of this document: the transcript is written **after** the stream finishes, not during it. A turn interrupted mid-stream leaves no row behind.

Home (Chat)

Home is the tool-light conversation surface: ask, brainstorm, search the web, read a link, draw a picture or cut a film. It deliberately cannot touch your files — that is Code's job — and it runs in its own router (`/v1/chat`) with its own transcript store, so a fault in the agent console cannot reach it.

The turn. `POST /v1/chat/sessions/{id}/messages` opens a Server-Sent Events stream. Tokens arrive as they are produced; tool calls, cited sources and errors arrive as named events on the same stream. `abort` cancels mid-turn, `regenerate` replays the last question, `edit` rewrites a question and re-runs from there, and `fork` copies a thread up to a chosen message so a different direction can be explored without losing the original.

The four tools. Home's agent gets `memory` (durable notes about you), `web_fetch` (read a URL), `make_video` (cut photos and clips into a film with ffmpeg) and, when an image deployment is configured, `generate_image` and `edit_image` on gpt-image-2. A system clause makes the distinction that matters: when somebody asks it to *make* something it makes the thing, rather than handing back the code that would make it.

The prompt library is the part with the most machinery, and it is served by three separate agents so that each answers the question it is actually good at:

Agent	Route	Reads	Answers with
`sharpen`	`POST /v1/chat/prompts/sharpen`	the conversation a prompt came from	one merged prompt + the turn numbers used
`compose`	`POST /v1/chat/prompts/compose`	the same conversation	the merged prompt **and** a verdict on every message — in or out, why, and the line it contributes
`refine`	`POST /v1/chat/prompts/refine`	the saved prompt alone	a clearer rewrite, the changed phrases marked, in one of three directions (clearer, shorter, more specific)

Saving from a message shows its working: a tray of every message in the session, each with the reason it was kept or dropped, and hovering a row lights the sentence it contributed. Unticking one re-joins the rest instantly, because the lines are already in hand. Editing a saved prompt swaps that offer for Refine, since a prompt in the library has no conversation behind it.

Voice uses Azure OpenAI Realtime over WebRTC. `POST /v1/chat/voice/session` mints a short-lived ephemeral key so the browser connects directly to Azure — audio never passes through Compass.

Work IQ is optional retrieval over Azure AI Search. When it is on, the answer carries a `work_iq_sources` event and the citations render under the reply.

Attachments are kept, not just read: photos, clips and recordings are written to blob under `chat/<session>/` so a film asked for three turns later can be cut from them.

Code (Agent Console)

Code is the module with hands. An agent runs against a real folder on disk, reads and edits files, runs shell commands, drives a browser, and asks permission before anything it cannot take back.

The tool catalogue — 18 tools. `file_read`, `file_write`, `file_edit`, `glob`, `grep` for the filesystem; `bash` and `bash_output` for a persistent shell whose working directory survives between calls; `browser` and `screenshot` for a real Chromium the agent can see;

`web_fetch`, `memory`, `todo_write`, `ask_user`, `consult`, `agent` (a subagent in its own sidechain), `find_tools` (used when the catalogue is too large to describe in full each request), and `generate_image` / `edit_image`.

Permission is a gate, not a setting. Every tool declares whether it is read-only. A write, a shell command or a browser navigation goes through a `PermissionBroker` with one of three policies — `interactive` (the person is asked, and the request streams to the UI as a card), `auto_grant`, or `auto_deny`. `POST /v1/sessions/{id}/permissions/{request_id}` carries the answer back into the waiting turn. Shell commands additionally pass a static analyser (`bash_security.py`) before they are offered.

The sandbox. On macOS and Linux the agent's shell runs inside an OS sandbox scoped to the session's workspace root. `COMPASS_SANDBOX=0` turns it off; on Windows, where no boundary exists, Compass warns once rather than pretending.

Workspaces are the folders Compass is allowed to work in. The module registers them (`POST /v1/workspaces/folder`), clones from GitHub, lists and searches files, shows `git status` and diffs, discards changes, opens the folder in VS Code and raises pull requests. Removing a workspace makes Compass forget the folder — nothing inside it is deleted.

Routines are scheduled agent runs: a saved prompt, a workspace and a cron-like trigger. Each run is recorded, and a finished run can fire a native OS notification. `GET /v1/routine-runs/recent` is what the shell polls to surface them.

Background tasks are long-running processes the agent spawned — a dev server, a test watcher. They outlive the turn that started them, stream their logs to `GET /v1/background-tasks/{id}/logs`, and can be stopped from the UI.

Artifacts. When the agent produces a self-contained document — a diagram, an SVG, an interactive page — it is rendered in a side panel rather than dumped into the transcript, inside a sandboxed iframe.

Design

Design turns a brief into a finished HTML document you can look at, edit in place and export. Fourteen templates set the brief the model is given: `flier`, `slides`, `mockups`, `mobile`, `email`, `animation`, `blank`, `wireframe`, `diagram`, `document`, `resume`, `research`, `object3d` and `colortype`.

The canvas is a sandboxed iframe (`sandbox="allow-scripts allow-popups"`) fed by `srcdoc`, with an editor agent injected into the preview and never into the stored markup. The opaque origin is a real constraint, not a detail: a cookie is withheld from requests

made inside it, which is why generated pictures are served from an unauthenticated route whose names are 128-bit random ids.

Generated artwork. Seven templates draw their own images with gpt-image-2 — flier, slides, mockups, mobile, email, animation, blank. Seven deliberately do not: a wireframe, a diagram, a document, a résumé, research, a 3D object and a colour study are all worse with a photograph in them. The model writes `data-draw="..."` placeholders into its markup and `artwork.fill()` replaces each with a drawn picture, capped at six per design and rendered at medium quality — measured on this deployment at low 24s, medium 37s, high 112s, and six at high would add two minutes to a document that already takes minutes.

Design systems are reusable looks: a palette, type and tokens, imported from pasted text, a stylesheet, a URL or a repository, and distilled into a system a project can be based on.

Pages, versions and files. A project holds pages; every save keeps a version that can be restored; and each project has its own folder of assets, scraps and uploads — now written through to blob so the project opens complete on any machine.

Comments are pinned to elements in the rendered design, and `POST /v1/design/projects/{id}/element` edits one element in place rather than regenerating the page.

Export produces a self-contained HTML file; thumbnails are rendered server-side with headless Chromium and cached.

Pipelines

Pipelines wires Compass's own capabilities together and runs them on a trigger — a directed graph of nodes with typed ports, executed by an engine that records every run.

Node types fall into four families:

Family	Nodes	Purpose
Flow	`flow.start`, `flow.if`, `flow.filter`, `flow.foreach`, `flow.wait`, `flow.approval`, `flow.set_variable`, `flow.fail`	branching, looping, waiting, human approval
Connectors	`connector.http`, `gmail.trigger`, `list_messages`, `get_message`, `send`, `list_issues`, `create_issue`, `list_pulls`	reaching outside — HTTP, Gmail, GitHub

Family	Nodes	Purpose
Database	`mssql.query`, `mssql.insert`	SQL Server read and write
Tools	any agent tool, exposed as a node	the same catalogue Code uses

Runs. `POST /v1/pipelines/{id}/run` streams progress as SSE; each node records its status, output, attempts and timing. A run can be cancelled, resumed after an approval node, or re-run a single node. History is partitioned by pipeline, so one pipeline's runs are one partition.

Connections hold the credentials a connector needs. They support OAuth sign-in with a callback route, a test call before you rely on them, and are checked for ownership on every use — a connection someone else owns is reported as absent.

The secret store is an interface with two implementations: `LocalSecretStore` (a file) and `KeyVaultSecretStore` (Azure Key Vault). It falls back to local **loudly** when Key Vault is asked for without a vault URL, because a silent downgrade to a weaker store is the failure nobody notices until it matters. This is the one piece of Compass still on local disk in the current deployment.

The builder can be generated: `POST /v1/pipelines/{id}/build` streams a model turn that assembles the graph from a description.

Capabilities are granted at edit time and checked twice — by the engine before a handler is called, and by the handler itself.

Estimate

Estimate answers two questions about a proposed system: should it be built with AI at all, and what does the first year cost. A model reads the brief and labels it; **every figure after that is computed**, which is the design decision the whole module rests on — the boundary between "a model was involved" and "arithmetic" is kept as a boundary.

Eight stages, streamed to the UI as they run so a costing that takes a minute reads as progress:

1. **classify** — read the brief, label each use case
2. **architect** — choose the delivery platform that fits
3. **feasibility** — score AI necessity, agentic fit and traditional fit
4. **cost** — development, infrastructure and tokens
5. **tokens** — project volume and model mix
6. **compare** — this against building it the standard way
7. **roi** — benefit, payback and three-year value

8. **report** — compose the document

What comes out: a verdict (*build with AI*, *hybrid*, or *build it the standard way*), a feasibility score out of 100, an itemised first-year cost, a token projection, a side-by-side comparison with the conventional build, and a payback period — with the confidence in each stated rather than implied.

Rate cards are per-owner (`PUT /v1/estimates/rate-card`), so a team's own day rates drive the arithmetic instead of defaults.

Exports are produced server-side as Excel and PDF, and a BRD can be generated from the same brief.

Missions

A mission is a long build that runs itself: a goal, a budget and a workspace, worked through session by session until the features are done or the money runs out. It is the only module where Compass decides what to do next without being asked each time.

The supervisor loop. `next_step()` reads the mission's state and chooses: plan, build, or review. Each session runs with a persona, and the split is enforced in code rather than in the prompt — a **builder** gets `mission_claim` and never sees `mission_verdict`, so it cannot mark its own work done however convinced it is; a **reviewer** gets the verdict tool and no way to write a file at all, because an evaluator that can edit the code it is grading fixes the bug instead of reporting it; a **planner** writes the plan and claims nothing.

Budget. $25 by default. Spend accumulates across sessions, and the UI projects how far the remaining budget reaches — "this budget reaches about feature 6 of 50".

Stall handling. Consecutive failures are counted and backed off exponentially. After a stall limit the mission pauses and says why, with the last error shown — in practice usually a model rate limit. *Continue anyway* overrides it.

Live progress streams over SSE (`GET /v1/missions/{id}/stream`), and a reopened browser replays what it missed rather than hanging on a stream that will produce nothing.

The workspace is never deleted. Deleting a mission removes its record and its session history; the folder it built stays exactly where it is on disk, and the deletion audit records the path so the decision is explicit rather than silent.

Business functions

The seventh module, and the only one that is not a general capability. The other six give everybody the same tool; this one mounts a set of claims about how one firm runs itself — which applications Finance offers, who may approve leave, when a period is closed. It is

off unless `COMPASS_BUSINESS_FUNCTIONS=1`, and off it imports nothing, mounts no routes and shows no tab.

A declarative catalog. One folder per business function under `compass/businessfunctions/catalog/`, each holding a `manifest.yaml`. The manifest owns the **words and the shape** — the name, the blurb, the scope selectors and their choices, every action's label and the sentence a person reads before confirming it, every rule's wording, which personas see the screen and which may act. Registered Python owns the **data and the judgement** — what the figures are, which rows exist, whether a rule applies right now. A feature names its handler by key and never by import path, because a dotted path in a YAML file is a YAML file that can import any module on the box. Adding a feature is a module plus an entry in some function's manifest; neither is enough alone, which is the point.

Three functions ship. Finance holds Form 26, a credit-statement reconciliation. Talent holds LMS, a leave queue. SCS — Supply Chain Services — runs one service, RewardLens, whose five sub-services together are the gift compliance system described below, and is where the module's design was worked out.

The assistant rail sits beside every feature. It reads the sentence, resolves which rows it means against what is actually on screen, and returns a **proposal** — nothing has changed. A person reads what would happen, the rows light up, and `confirm` performs it once. The gap between deciding and acting is the feature: a single call that both resolved and performed would be the same design with the safety removed. A proposal belongs to one person, expires after ten minutes, is single-use, is re-checked against current state at confirm, and an outward action covering several rows is never proposed at all. The model never chooses rows and never performs; it phrases facts that were already gathered, which makes a wrong answer a clumsy sentence rather than a wrong row.

A second agent answers, and cannot act. `chat.py` sits beside the proposal rail and is deliberately separate from it: no confirm path, no proposal store, no handler call that writes. It reads and it answers. Its brief is built by asking every sub-service for its figures and rows **through the reader's own scope, across every period** rather than only the tab on screen — so a question about last year is answered from last year, an employee's from their one row and Leadership's from the firm, and the model is never handed anything the person asking could not have read themselves. The brief is capped at 300 rows a screen and 90,000 characters, and the cap is said out loud inside it when it bites, so the model answers "the first 300 of 900" rather than reading a cut as the whole. A question the brief cannot answer gets "that is not on these screens": a model asked about gift policy will invent a plausible threshold, and a plausible wrong threshold is worse here than no answer, because somebody would act on it.

Two-layer tool grant. A function declares the tools its features may hold; a feature narrows that list and can never widen it. Above both sits a withheld set — bash, the filesystem, the browser, `web_fetch`, `agent` — that no business function may hold whatever its manifest says. The assistant here reads one screen and operates it; a shell and the open web are both ways out of that.

SCS: the gift compliance system

The firm always knew what it spent on gifts. What it could not say was that one person received ₹8,000, then ₹5,000, then ₹7,000 from three different teams and is now at ₹20,000 — past the point where Finance has to look. Nobody was hiding anything; the three purchases simply never met. Five screens close that, and the whole module exists at the scale of that one sentence.

Three levels, not a flat list. SCS is the business function, **RewardLens** is the one service inside it, and the five screens are its sub-services. The level earns its place: without it, five screens read as five services. The sidebar names RewardLens alone; opening it shows Dashboard, Gift requests, Tracker, Tax & budget and Audit trail as cards. The manifest carries the shape — a `services:` block, and a `service:` on each feature — and anything no service claims comes last in an unnamed group, so a mistyped `service:` shows up as a feature sitting outside rather than a feature that vanished. A row opens in a sheet: everything it holds and every action it allows in one place, rather than a caret and a scatter of buttons down the table.

The Tracker is the register, and the screen that follows an order through the chain. Its first tab is every order this person is involved in and how far each has got — raised by them, waiting on their desk, about them, or already signed by them — so a desk sees its own work rather than the whole firm's. On the remaining tabs the unit is a **person, not a purchase**: recipients and their running total against the annual limit, with the items underneath. A list of purchases is what the firm already had. The arithmetic is addition and nothing else: the limit comparison is `>`, the excess is a subtraction, and no part of a figure a tax decision rests on is generated. Employees declare what they were given outside Procurement; compliance reviews those declarations; a correction supersedes rather than edits.

Gift requests is the half that happens first, and it is a *chain*: a request sits with exactly one desk at a time, and signing off is what moves it. Service Line Approver → Finance → Payroll → Procurement / SCS → Accounts Payable → back to the requester to record what was actually given → to the recipient to confirm receipt → closed. Each desk sees what is waiting on it, and nothing behind it moves until it acts. Before the stage model a request had a *state* and every action was gated on a role, which produced a sequence by accident: anybody holding compliance could approve something that had never been near Payroll.

The stage is **folded from the trail, never stored**, so a stage cannot disagree with the events that moved it. Routing is one question — is this stage owned by a persona you hold — and two steps invert it rather than bend it. **Acknowledgement** is only ever about yourself: confirming you received something is not deciding your own case. **Distribution** is only ever the person who raised it, because they handed it over and are the only one who can say what differed. Held to the ordinary not-your-own rule, distribution was unreachable — the requester was locked out of the one step the business's own persona sheet gives them.

Five desks CHECK; the last two RECORD, and only a check may refuse. Service Line, Finance, Payroll, Procurement and Accounts Payable can each send a request back with a reason. Distribution and acknowledgement cannot: by then the gift has been approved, bought and paid for, so the requester's job is to say what was actually handed over rather than to reopen a decision five other desks already took — and a recipient able to decline their own gift would have a way to make a record disappear, which is the opposite of a control. Anybody may still raise a request — a Business Requester whose job it is, or an employee thanking somebody — and the person who raised it may withdraw it while it is still being checked.

Three checks run **before** the decision and are shown to whoever signs off: policy, budget, and what it does to the recipient's year. The last is RewardLens's own arithmetic asked one gift earlier, and it is the point of the system: "takes them to ₹26,000 of ₹15,000 — ₹11,000 over" is a sentence said while saying no still costs nothing. Policy and budget **block**; a tax consequence does **not** — it is shown, and the sign-off records that the signer knew. Refusing automatically would be a machine making a judgement that belongs to a person, and would teach everybody to route around the form.

What a desk is asked to **write down belongs to the stage, not to the button**. Finance and Payroll must say what they checked; the requester must say what was actually given and how it differed; the other five ask nothing, because there the click is the act. Hung off the action instead, one note label covered all seven steps — and the recipient confirming a hamper had arrived was asked "What did you check?" and could not get past it.

Two reports, because they answer different questions. The *order report* is one order with every step and who took it: the half a screen cannot show, since a screen has one line per order and this has one line per step. The *consolidated report* is cut on the day the **recipient confirmed receipt** — not when it was asked for, approved or bought, which can be months earlier and omits exactly the orders that crossed the boundary. Both are built from the same handler the screen uses, so there is no second path to the data, and both say on the sheet which window they were cut on.

The Dashboard is leadership's only view of the whole position, and the one screen in the section that draws rather than lists: four charts — annual budget against what each team has spent, where the spend goes, everybody past ₹50,000 this year, and the amount above the limit by team. Each is clickable through to the rows it was drawn from, because a chart nobody can get behind is a chart nobody can check. The charts are hand-rolled SVG over the same tabs the grid reads, so the picture and the table cannot disagree. **Tax & budget** is Finance's: who has been referred, everybody past the limit whether referred or not, and what each team has committed against its budget. **Audit trail** is the raw history, one row per thing that happened, newest first, with a name and a time against it — and no actions at all.

Thresholds are versioned, not edited. The annual limit, the single-gift threshold, what counts towards the limit and the team budgets live in `compass/businessfunctions/policy.py` as the one source of truth every policy check, tax sentence and budget refusal reads. They were Python constants, which meant the firm's own compliance rule could only be changed by deploying, and that nobody could say what the rule had been last March. Setting a new one now does not change the old one: it appends a version with the moment it takes effect. `current()` is what a new record is judged against and `at(when)` what an old one was, so a gift that was within the limit on the day it was given does not become a breach because somebody lowered the limit afterwards — and a breach does not stop being one because somebody raised it. Only the `admin` persona may set a version: it is the highest-leverage change anybody can make here, so it is deliberately not in the hands of the teams whose own work it judges. Every version carries who set it and why, and lands on the same audit trail as every other decision.

Roles, and how much of a screen is yours

The useful question about a persona is not whether they may open a screen but **how much of it is theirs**:

- `self` — their own record, and nothing about anybody else
- `team` — the people who report to them
- `firm` — everybody

Most of the eight personas on the business's sheet want the same screen at a different breadth, so breadth is the model and a boolean is not. An employee opening the register sees one row — their own year against the limit, what is left, what they still have to acknowledge — which is a useful screen rather than a locked door.

The eight are the business's own, not invented here: Business Requester, Service Line Approver, Finance Team, Payroll Team, Procurement / SCS, Accounts Payable, Employee, and Leadership / Compliance. **Auditing folds into Leadership / Compliance** rather than

becoming a ninth, because that is where the sheet puts it — a role nobody in the firm holds is a role nobody can be held to, and a login invented for it showed empty screens while the checks against it passed for the wrong reason.

The four desks that own a step in the chain but have no business reading the firm's register — Procurement, Accounts Payable, the requester, and the recipient — see the *chain* at firm breadth and the *register* only at `self`. They need the order in front of them, not everybody's year.

Who somebody **is** lives in one place, `compass/businessfunctions/people.py`: a directory standing in for the real one, holding the employee record, the address and the roles. What that lets them see is declared **per feature in its manifest**, and what it lets them do **per action**. The same person is firm-wide on one screen and self-only on another without the directory changing, which is the point — a role carrying its own permissions would have to be re-granted every time a feature was added. A feature that declares no visibility is unrestricted, so Form 26 and LMS behave exactly as they did before any of this existed.

The narrowing happens **once**, where the rows are built, so a feature's figures, tabs, table, export, assistant answers and monthly report all narrow together. A filter applied on the way out would leave the counts above the table describing a firm the reader cannot see.

Roles are resolved server-side from the signed-in identity and never accepted from a request. The resolution follows `identity_aliases` back from the canonical identity to the login, because a deployment that aliases `admin` to an address otherwise matches nobody — every screen renders, every figure is right, and the two rules that stop a person deciding their own record silently never fire.

The register and the trail

`compass/businessfunctions/ledger.py` keeps two things apart on purpose. The **register** is what the firm gave and to whom: one record per item, written once and never altered. The **trail** is what people did about those records — reviewed, queried, answered, superseded, referred, signed off, acknowledged, closed — appended and never edited.

That separation is not tidiness. The invariant the whole feature defends is that **a record does not change after the fact**: a correction supersedes, a breach does not un-receive a gift, and a total is the sum of what was received so no department edits it. A store that let a decision write back onto an item would quietly undo that, with no sign but an audit that stops reconciling.

So state is a **fold**. Whether something is accepted, queried, answered or superseded is read off what happened to it, in order. There is one way for a fact to be true — something made it true, and that something is still there. Re-querying an item does not clear the

previous answer; an answer given before the new question was asked is already not a reply to it, and the old one stays where an auditor can see it.

Handlers are synchronous and the store is not, so writes land in memory and are flushed by the route that caused them, after the work rather than before — a slow store delays a reply instead of undoing a decision. The cost, plainly: a crash in the sub-millisecond window between the two loses the last decision. The fix is an async feature contract, worth doing when that contract is next opened.

Seeding happens only into an **empty** register, so a fresh deployment gets a sample and one with a year of declarations keeps them. Assistant proposals are deliberately **not** persisted: a plan is seconds of intent, and one that outlived the screen it was made against would be confirmed against rows that have moved.

Notices, exports and monthly reports

A notice is a **record, not a side effect**. "We asked them on the 9th" is something a compliance screen has to be able to say, and it cannot if the only trace is a line in a mail server's log. Every notice is written down first — who it was for, what it said, which row it was about — and delivery is an attempt made against that record. With no mail server configured the notice is **held** and the row says so, because a reviewer who believes somebody was emailed and nobody was is the failure this prevents. It goes to the person and nobody else: `send_email` falls back to the operator's own address when given no recipient, so every call passes one explicitly and a notice with no address is held.

The export is the screen. It asks the handler for the same figures and rows the surface asks for, through the same scope — not a second query with its own filters, which is how a spreadsheet comes to say four people are over while the screen says three. Two things follow: it cannot be a second path to the data, so somebody who is shown nothing gets an empty workbook; and it cannot drift, so it already exports every shipped feature without knowing what any of them contain. The summary sheet carries the period, the moment, who asked and a line saying the figures move, because a spreadsheet outlives the screen it came from.

Monthly reports are a standing request a person makes for themselves. There is no recipient field and no route for stopping somebody else's: a list would mean one person's click causing mail every month to somebody who never asked and may not be allowed to read it. The report is rendered through the subscriber's own scope, so losing a role empties next month's report — not because anything checks, but because it is the same code path that would empty their screen. It sends the figures and a pointer, not the workbook: attaching it would put every employee's gift record in an inbox on a schedule.

What is a stand-in

Three things here are deliberately small enough to be obvious rather than half-built, and each is where a real integration goes: the **directory** (roles, names and addresses are a dict in code), the **budgets** (one number per team), and the **policy rules** (three rules written out, not configurable). Fixture email addresses are all under `.invalid`, which RFC 2606 reserves so they can never resolve.

Out of scope by the requirement's own phasing: vendors, external recipients and hospitality aggregation are Phase 2; multiple countries and currencies are Phase 3.

Where the business value is

Each module earns its place differently. The mechanism matters more than the claim: what is actually removed from somebody's week.

Module	Who it serves	What it replaces	Where the return comes from
Home	everyone	ad-hoc ChatGPT use outside the tenancy	one governed surface — the conversation, its attachments and what it drew stay inside your Azure subscription
Code	engineers	manual edit–run–test cycles	an agent that reads the repo, makes the change, runs it and raises the PR; routines do the recurring parts unattended
Design	marketing, pre-sales, product	an agency round trip or a designer's queue for a flier, deck or mock-up	a first draft in minutes, editable in place, on your own design system
Pipelines	operations	scripts nobody owns and manual handoffs	scheduled, auditable automation with approval gates and a recorded history per run

Module	Who it serves	What it replaces	Where the return comes from
Estimate	pre-sales, delivery leads	a spreadsheet rebuilt per bid	a consistent, defensible costing in minutes, with the verdict and confidence stated
Missions	engineering leadership	a backlog nobody has capacity to start	a long build that proceeds unattended inside a fixed budget, with a reviewer that cannot mark its own work done
Business functions	compliance, finance, audit, procurement, leadership, every employee	spreadsheets and email threads per team, with no central view of what any one person received	a breach caught before the gift is bought rather than at the year end — and an audit trail that can show every approval happened, with a name and a time against it

Two cross-cutting effects. First, one identity and one storage layer means the audit question — *who made this, who deleted it, where is it* — has a single answer across seven modules. Second, every artefact is portable: a conversation, a design or an estimate opens on any machine the person signs into, because nothing of record lives on the box it was made on.

Where every piece of data lives

Two rules decide the split: **a record goes to Cosmos, a byte goes to blob.** Anything over roughly 2 MB is spilled to blob with a reference left in the document, because that is Cosmos's item limit.

Azure Cosmos DB — database `compass`

Container	Partition key	Holds	Written by
`chat`	`sessionId`	Home transcripts and thread metadata	Home
`transcripts`	`sessionId`	Code agent transcripts	Code
`transcripts-meta`	`owner`	Code session titles, ownership, model, mode	Code
`catalog`	`owner`	the small owner-scoped records: media index, prompts, prompt order, routines, missions, workspaces	all modules
`designs`	`owner`	design projects and systems	Design
`pipelines`	`owner`	pipeline definitions and connections	Pipelines
`runs`	`parentId`	pipeline and routine run history, partitioned by its parent	Pipelines, Code
`estimates`	`owner`	estimates and per-owner rate cards	Estimate
`users`	`id`	accounts	Auth
`bf_gift_item`	`owner`	the gift register: one record per item given, written once and never altered	Business functions
`bf_gift_event`	`owner`	the audit trail: append-only, one row per decision — reviewed, answered, superseded, referred, signed off, acknowledged, closed. State is folded from it rather than stored.	Business functions
`bf_gift_request`	`owner`	gifts asked for but not yet given — a proposal, not a record; nothing reaches the register until it is handed over	Business functions
`bf_notice`	`owner`	the outbox: what was meant to be sent, to whom, and whether it went or was held	Business functions
`bf_report`	`owner`	monthly report subscriptions and the figures each run reported	Business functions

Container	Partition key	Holds	Written by
`bf_policy`	`owner`	every version of the thresholds and team budgets, append-only, each with who set it, why, and the moment it took effect	Business functions
`audit`	`owner`	one row per deletion, expired by a per-item TTL	all modules

Azure Blob Storage

Container	Prefix	Holds
`compass-media`	`generated/`	pictures drawn or edited by gpt-image-2
`compass-media`	`screenshots/`	what the Code agent's browser saw
`compass-media`	`chat/<session>/`	a thread's uploads and the films cut from them
`compass-media`	`messages/<session>/`	message content spilled over the Cosmos item limit
`compass-design`	`pages/` , `versions/`	design markup and version history
`compass-design`	`files/<project>/`	a design's own assets, scraps and uploads
`compass-artifacts`	—	tool-result spills
`compass-jobs`	—	long-running job state

Still on local disk, by design

The agent's workspaces (`data/workspaces/`) — your actual code, which belongs on the machine. Everything else local is either a cache that rebuilds from blob on demand, or `data/secrets/connections.json` , the one record not yet in the cloud: Key Vault is implemented and needs only a vault URL.

The retention rule

Deleting removes the record **and** the bytes — the Cosmos row, the blob, and the working copy on disk. What survives is one audit row saying who deleted what, from which module, which session it belonged to and what went with it — never the content. Those

rows carry a `ttl` and Cosmos expires them itself, so retention needs no sweep. `COMPASS_AUDIT_RETENTION_MONTHS` sets it; the default is three months, `0` keeps them.

API reference

181 endpoints. Every one except the two byte routes noted below requires the session cookie via `Depends(require_user)`. The **Called from** column names the method in `frontend/src/app/compass-api.service.ts`, which is the only place the front end issues HTTP — components call that service, never `HttpClient` directly.

Auth — `compass/common/auth.py`

Method & path	Purpose	Store	Called from
`POST /v1/auth/login`	exchange credentials for the cookie	`users`	`AuthService.login`
`POST /v1/auth/logout`	clear the cookie	—	`AuthService.logout`
`GET /v1/auth/me`	who am I	`users`	`AuthService.restore`
`POST /v1/auth/me/name`	set display name	`users`	account menu
`GET /v1/auth/users`	list accounts	`users`	admin surface

Home — `compass/home/routes.py`, prefix `/v1/chat`

Method & path	Purpose	Store	Called from
`GET /sessions`	the sidebar list	`chat`	`listChatSessions`
`POST /sessions`	start or resume a thread	`chat`	`createChatSession`
`GET /sessions/{id}/transcript`	paged history	`chat` + blob spills	`chatTranscript`
`POST /sessions/{id}/messages`	**SSE turn**	`chat`, blob	`streamChatMessage`

Method & path	Purpose	Store	Called from
`POST /sessions/{id}/regenerate`	redo the last answer	`chat`	`streamChatRegenerate`
`POST /sessions/{id}/edit`	rewrite a question, re-run	`chat`	`streamChatEdit`
`POST /sessions/{id}/fork`	branch the thread	`chat`	`forkChatSession`
`POST /sessions/{id}/abort`	cancel mid-turn	—	`abortChat`
`PATCH /sessions/{id}`	rename, star	`chat`	`patchChatSession`
`DELETE /sessions/{id}`	delete thread + media + index rows	`chat`, blob, `catalog`, `audit`	`deleteChatSession`
`GET /sessions/{id}/media/{file}`	serve an upload	blob	`<img>` / `<video>` src
`POST /sessions/{id}/voice-turn`	a spoken turn	`chat`	voice mode
`GET /prompts`	the library + its order	`catalog`	`savedPrompts`
`POST /prompts`	save one	`catalog`	`savePrompt`
`PATCH /prompts/{id}`	edit one	`catalog`	`editSavedPrompt`
`DELETE /prompts/{id}`	delete one	`catalog`, `audit`	`deleteSavedPrompt`
`PUT /prompts/order`	the arrangement	`catalog`	`setPromptOrder`
`POST /prompts/sharpen`	merge a thread into one prompt	none — generation only	`sharpenPrompt`
`POST /prompts/compose`	the same, plus a verdict per message	none	`composePrompt`
`POST /prompts/refine`	rewrite a saved prompt	none	`refinePrompt`

Method & path	Purpose	Store	Called from
`GET /work-iq`	is retrieval configured	—	`workIqStatus`
`GET /voice`	is realtime configured	—	`voiceStatus`
`POST /voice/session`	mint an ephemeral WebRTC key	—	`voiceSession`
`GET /favicon`	cached site icon for a citation	local cache	citation strip

Code — `compass/code/routes.py`

Method & path	Purpose	Store	Called from
`GET /v1/sessions`	session list	`transcripts-meta`	`listSessions`
`POST /v1/sessions`	new session	`transcripts-meta`	`createSession`
`GET /v1/sessions/{id}/transcript`	history	`transcripts` + blob	transcript loader
`POST /v1/sessions/{id}/messages`	**SSE agent turn**	`transcripts`, blob	`streamMessage`
`POST /v1/sessions/{id}/messages/{uuid}/edit`	edit and re-run	`transcripts`	`streamEdit`
`POST /v1/sessions/{id}/regenerate`	redo	`transcripts`	`streamRegenerate`
`POST /v1/sessions/{id}/permissions/{req}`	answer a permission prompt	—	`resolvePermission`
`POST /v1/sessions/{id}/questions/{req}`	answer `ask_user`	—	`answerQuestion`
`POST /v1/sessions/{id}/suggest`	next-step suggestions	—	`suggestNext`
`POST /v1/sessions/{id}/fork` · `abort`	branch · cancel	`transcripts`	`forkSession` · `abort`

Method & path	Purpose	Store	Called from
`PATCH` · `DELETE /v1/sessions/{id}`	rename · delete + purge media	meta, blob, `audit`	`updateSession` · `deleteSession`
`GET /v1/workspaces` · `POST /v1/workspaces/folder`	list · register a folder	`catalog`	`listWorkspaces` · `addFolderWorkspace`
`DELETE /v1/workspaces/{id}`	forget the folder (files untouched)	`catalog`, `audit`	`deleteWorkspace`
`GET /v1/workspaces/{id}/files` · `file` · `files/search`	browse · read · search	disk	`listFiles` · `readFile` · `searchFiles`
`GET /v1/workspaces/{id}/git` · `diff`	status · diff	git	`gitStatus` · `gitDiff`
`POST /v1/workspaces/{id}/discard` · `pr` · `open-in-vscode`	revert · raise PR · open editor	git, GitHub	`discardChanges` · `createPr` · `openWorkspaceInVsCode`
`POST /v1/github/clone` · `GET /v1/github/repos`	clone · list repos	disk, GitHub	`githubClone` · `githubRepos`
`GET` · `POST` · `PATCH` · `DELETE /v1/routines...`	scheduled runs CRUD	`catalog`, `runs`	`routines`, `createRoutine`, `updateRoutine`, `deleteRoutine`
`POST /v1/routines/{id}/run` · `GET /v1/routine-runs/recent`	run now · poll finished	`runs`	`runRoutineNow` · `recentRoutineRuns`
`GET /v1/background-tasks` (+ `logs`, `stop`, `clear`)	spawned processes	in-process	`backgroundTasks`, `backgroundTaskLogs`, `stopBackgroundTask`
`POST /v1/pick-folder`	native folder picker	—	`pickFolder`

Design — `compass/design/routes.py`

Method & path	Purpose	Store	Called from
`GET /v1/design/templates`	the 14 templates	—	`designTemplates`
`GET` · `POST /v1/design/projects`	list · create	`designs`	`designProjects` · `createDesign`
`GET` · `PATCH` · `DELETE /v1/design/projects/{id}`	open · rename · delete	`designs`, blob, `audit`	`designProject` · `patchDesign` · `deleteDesign`

Method & path	Purpose	Store	Called from
`POST /v1/design/projects/{id}/generate`	**SSE** generate the design, draw its artwork	`designs`, blob, `catalog`	design chat
`POST /v1/design/projects/{id}/html`	save edited markup	blob	`saveDesignHtml`
`POST /v1/design/projects/{id}/element`	edit one element in place	blob	`editElement`
`GET` · `POST` · `DELETE .../pages`	the project's pages	blob	`designPages`, `addDesignPage`, `deleteDesignPage`
`GET .../versions` · `POST .../versions/{v}/restore`	history · restore	blob	`designVersions` · `restoreDesignVersion`
`GET` · `POST` · `DELETE .../files` (+ `files/read`)	the project's own folder	blob + disk cache	`designFiles`, `writeDesignFile`, `deleteDesignFile`
`POST` · `DELETE .../comments`	pin and remove comments	`designs`	`addDesignComment` · `deleteDesignComment`
`GET .../thumbnail` · `.../export`	rendered thumb · standalone HTML	cache · blob	`designThumbUrl` · `designExportUrl`
`GET` · `POST` · `DELETE /v1/design/systems...`	design systems CRUD, setup, duplicate, usage	`designs`	`designSystems`, `createDesignSystem`, `setUpDesignSystem`
`POST /v1/design/clarify` · `/attach`	sharpen a brief · keep an attachment	blob	`clarifyDesign`

Pipelines — `compass/pipelines/routes.py`

Method & path	Purpose	Store	Called from
`GET` · `POST /v1/pipelines`	list · create	`pipelines`	`pipelines` · `createPipeline`
`GET` · `PATCH` · `DELETE /v1/pipelines/{id}`	open · save · delete + its runs	`pipelines`, `runs`, `audit`	`getPipeline` · `savePipeline` · `deletePipeline`
`POST /v1/pipelines/{id}/run`	**SSE** execute the graph	`runs`	`runPipeline`
`POST /v1/pipelines/{id}/nodes/{node}/run`	run one node	`runs`	`runPipelineNode`

Method & path	Purpose	Store	Called from
`POST /v1/pipelines/{id}/validate`	what is wrong with this graph	—	`validatePipeline`
`GET` · `POST` · `DELETE /v1/pipelines/{id}/build`	**SSE** build the graph from a description	`pipelines`	`streamBuild` · `resetBuild`
`GET /v1/pipelines/{id}/runs` · `GET /v1/pipeline-runs/{run}`	history · one run	`runs`	`pipelineRuns`
`POST /v1/pipeline-runs/{run}/cancel` · `resume`	stop · continue past approval	`runs`	`resumePipelineRun`
`GET` · `POST` · `PATCH` · `DELETE /v1/pipeline-connections`	connection CRUD	`pipelines` + secret store	`pipelineConnections`, `createConnection`, `deleteConnection`
`POST .../{id}/test` · `sign-in` · `GET .../oauth/callback`	verify · OAuth	secret store	`testConnection` · `signInConnection`
`GET /v1/pipelines/node-types` · `/v1/pipeline-connection-types`	the palettes	—	`connectionTypes`

Estimate — `compass/estimate/routes.py`

Method & path	Purpose	Store	Called from
`GET` · `POST /v1/estimates`	list · create	`estimates`	`EstimateApi.list` · `create`
`GET` · `DELETE /v1/estimates/{id}`	open · delete	`estimates`, `audit`	`EstimateApi.get` · `remove`
`POST /v1/estimates/stream`	**SSE** the eight stages	`estimates`	estimate runner
`POST /v1/estimates/preview` · `draft` · `brd`	platform preview · draft · generate a BRD	—	estimate wizard
`GET /v1/estimates/{id}/export/excel` · `pdf`	downloads	—	export buttons

Method & path	Purpose	Store	Called from
`GET` · `PUT /v1/estimates/rate-card`	per-owner day rates	`estimates`	rate-card editor
`GET /v1/estimates/catalog`	archetypes and defaults	—	wizard

Missions — `compass/missions/routes.py`, prefix `/v1/missions`

Method & path	Purpose	Store	Called from
`GET` · `POST`	list · create	`catalog`	`missions` · `createMission`
`GET /{id}`	one mission with features	`catalog`	`mission`
`POST /{id}/run` · `GET /{id}/stream`	**SSE** start · watch	`catalog`, `transcripts`	`streamMission`
`POST /{id}/abort` · `DELETE /{id}`	stop · delete (workspace kept)	`catalog`, `audit`	`abortMission` · `deleteMission`

Business functions — `compass/businessfunctions/routes.py`, prefix `/v1/business-functions`

Fifteen endpoints, mounted only when `COMPASS_BUSINESS_FUNCTIONS=1`; switched off, none of them exists in the route table. Every one takes `Depends(require_user)`, and every one that reads or writes records loads the register first and saves after. The scope — entity, period, **and the caller's roles and breadth** — is assembled on the server from the signed-in identity and the feature's own manifest; no request body can name who the caller is or how much they may see.

Method	Path	Purpose	Store	Called from
GET	`/v1/business-functions`	The switcher: every loaded function, its subtitle and how many features it holds.	catalog on disk	`business-functions.ts` → `bf-api.ts`

Method	Path	Purpose	Store	Called from
GET	`/{function_id}`	One function's overview: its features and a "needs you" queue built from each feature's own warning figures, at that person's breadth per feature.	`bf_gift_*`	`openFunction`
GET	`/{function_id}/features/{feature_id}`	The workbench: figures, tabs, rows, the rules in force, the row key, the scope choices and the rail. Returns `needs_scope` instead when a selector is unchosen.	`bf_gift_*`	`openFeature`, `refresh`
POST	`/{feature}/act`	One action against rows the person pointed at. The click is the confirmation; the handler re-checks state, the role, a closed period and the conflict rules. `note` carries a reason where the manifest asks for one.	`bf_gift_event`	`rowAct`
POST	`/{feature}/ask`	The rail. Changes nothing: returns a **plan**, an **answer** carrying facts for a model to phrase, or a **clarification**. Resolves against the open tab.	—	`send`
POST	`/plans/{plan_id}/confirm`	Carry out a plan, once, for the person who made it. Re-checked against current state.	`bf_gift_event`	`confirmPlan`
POST	`/plans/{plan_id}/cancel`	Drop a plan and say plainly that nothing happened.	—	`cancelPlan`
POST	`/{feature}/forms/{form_id}/preview`	What a filled-in form **would** do — the resulting total, the three request checks. Changes nothing.	—	`checkForm`
POST	`/{feature}/forms/{form_id}`	Record it. A declaration lands against the signed-in person; an answer lands against the row it is about. The body cannot name whose record to write to.	`bf_gift_item`, `bf_gift_request`, `bf_gift_event`	`submitForm`

Method	Path	Purpose	Store	Called from
GET	`/{feature}/export.xlsx`	The screen as a workbook: a summary sheet with provenance, then one sheet per tab. `?since=` / `?until=` cut it to a date window, which the summary sheet names along with which date it is about; `?row=` returns one record instead, with every event that happened to it — the order report. Built from the same handler calls the screen makes, so it inherits the same access rules.	—	`download`
GET	`/{feature}/report`	This person's own monthly report for this screen, and what it has sent.	`bf_report`	`loadReport`
PUT	`/{feature}/report`	Ask for it monthly, for yourself. No recipient field exists.	`bf_report`	`toggleReport`
DELETE	`/{feature}/report`	Stop your own. There is no route for stopping anybody else's.	`bf_report`	`toggleReport`
POST	`/{feature}/report/send`	Send yours now, through the same render and delivery the scheduler uses.	`bf_report` , `bf_notice`	`sendReportNow`
POST	`/{function_id}/ask`	The function assistant. Answers in prose from a brief built across every sub-service the reader may open, at their own breadth and over every period — not only the tab on screen. Reads only: no plan, no confirm, no write.	every bf_* store, read-only	`askFunction`

A background loop started with the module checks each subscription against the clock once a minute and sends what is due. Monthly is not one of Routines' trigger types, so the day-of-month arithmetic is here while the timezone handling is reused — the 31st of a short month fires on its last day rather than skipping it, and the grace window is a day so a restart on the morning of the 1st delays a report instead of losing it.

Shared — `compass/common/routes.py`

Method & path	Purpose	Store	Called from
`GET /healthz`	liveness, models, capabilities	—	`health` on boot
`GET /v1/models`	deployments and effort ladders	—	model picker
`GET /v1/media`	everything this person has made	`catalog`	media browser
`GET /v1/media/generated/{name}`	serve a drawn picture — **unauthenticated by design**	blob	`<img>` in chat and the design canvas
`GET /v1/screenshot-cache/{id}`	serve an agent screenshot — **auth + ownership**	blob	`<img>` in the Code transcript
`POST /v1/screenshot`	capture a URL	blob	`screenshot`
`GET` · `POST` · `PATCH` · `DELETE` `/v1/memory`	durable notes about the user	`catalog`	`listMemory`, `patchMemory`, `deleteMemory`
`GET /v1/recap`	what happened in the last N days	several	`recap`
`GET /v1/customize`	per-user preferences	`users`	`customize`
`POST /v1/speech`	text to speech	—	`synthesizeSpeech`

The two byte routes are deliberately different. `/v1/media/generated/{name}` has no auth because it is read by the design canvas, a sandboxed iframe with an opaque origin that a SameSite cookie is withheld from; its names are 128-bit random ids, so the URL is the credential — unguessable but copyable, and nothing should be put there that only the login protects. `/v1/screenshot-cache/{id}` has both auth *and* an ownership check, because a screenshot is whatever the agent's browser was looking at and its id is only twelve hex characters.

Identity, ownership and deletion

The token. `POST /v1/auth/login` compares the password with `hmac.compare_digest` — constant time — and returns a signed token as an httpOnly cookie: `base64(payload).base64(hmac_sha256(payload, secret))`, where the payload is `{"u": username, "exp": unix_seconds}`. There is no server-side session; `require_user` verifies the signature and expiry on every request and is a dependency on all 136 stateful routes. Accounts come from `COMPASS_AUTH_USERS` as one comma-separated list — `alice:pw1,bob:pw2`. That variable **replaces** the whole account map; a second variable with another name is not read.

The ownership rule lives in one file, `compass/common/ownership.py`, so it cannot drift between modules. Two functions: `owner_for(user)` stamps a record on creation, and `visible_to(user, owner)` decides a read. It **fails open on an empty owner** — a record written before ownership existed is visible to everyone. That is deliberate and right for a single-account install, where the alternative looks exactly like data loss, and wrong the moment a second person signs in. The migration scripts in `scripts/` exist to close that gap by naming the owner, rather than by changing the rule.

One module is stricter: the **prompt library** matches the owner exactly, because a saved prompt is written in the first person about somebody's own day, and showing one to a stranger is a different kind of wrong from showing them a shared pipeline.

Not yours reads as not there. Every per-record route answers 404 rather than 403 when a record belongs to someone else. Whether a given id exists is itself something a person who cannot see it has no business learning.

Deleting cascades. Removing a conversation takes its transcript, its uploads, the pictures drawn in it, the screenshots taken in it and their index rows — the Cosmos rows, the blobs and the working copy on disk. Deleting a pipeline takes its whole run history. Deleting a mission leaves the workspace it built on disk, and says so in the audit row.

And it is recorded. One row per deletion in the `audit` container, partitioned by owner: who, what, which module, which session, when, and a tally of what went with it — never the content, because the content is the thing that was deleted. Writing the note can never cost a deletion: every call swallows its own failures, so somebody who asked for something to be removed has it removed whether or not the log was reachable.

In the browser, signing out clears everything the shell holds on someone's behalf before the next person arrives, and the same clearing runs on sign-in, because a session that expires never calls sign-out. Three per-browser preferences remain in `localStorage` — theme, notification sound, and whether the pipeline builder panel is open. They hold no data.

For a new developer

Layout

```
compass/
  api/server.py        the FastAPI app — routers, lifespan, middleware
  common/              72 files: auth, config, ownership, audit, persistence,
                       the model gateway, the shared tool base, media index
  home/                Chat: engine, prompts, media, video, vision, favicons
  code/                Agent console: engine, routines, workspaces, tools/
  design/              projects, systems, markup, artwork, export, files
  pipelines/           engine, nodes/, store, secrets, credentials, oauth
  estimate/            pipeline (8 stages), engine, catalog, exports
  missions/            supervisor, engine, runner, personas, artifacts
  businessfunctions/   manifest, registry, catalog/ (one folder per
function),
                       features/ (registered handlers), people, ledger,
                       policy (versioned thresholds), assistant (proposes),
                       chat (answers), notices, reports, export
frontend/src/app/      Angular 21, standalone + signals, zoneless
scripts/               one-off migrations, each with a --apply gate
```

Five conventions that explain most of the code

1. **One file owns a rule.** Ownership lives in `ownership.py`, deletion notes in `audit.py`, the tool contract in `tools/base.py`. If a rule appears twice, that is a bug waiting to happen.
2. **Comments say *why*, not *what*.** The codebase explains the decision and often the failure that caused it. When you change behaviour, update the reasoning beside it — a stale *why* is worse than none.
3. **Best-effort never blocks the main job.** Index rows, audit notes and blob mirrors all swallow their own failures. A picture whose index row failed is still drawn, stored and shown.
4. **Optimistic UI with rollback.** The row leaves the list before the request does; on failure the previous state is restored exactly and the error names the server's own reason.
5. **Migrations are scripts, not startup code.** Everything in `scripts/` dry-runs by default and needs `--apply`. None of them delete the source.

Running it

```
# backend — reads .env from the repo root
uvicorn compass.api.server:app --port 8000

# frontend — proxies /v1 to :8000
cd frontend && npm start        # http://localhost:4200
```

The variables that decide behaviour: `COMPASS_STORAGE_BACKEND` (`local` or `cosmos`), `AZURE_COSMOS_ENDPOINT` / `_KEY`, `AZURE_STORAGE_CONNECTION_STRING`, `AZURE_OPENAI_ENDPOINT` / `_KEY` / `_DEPLOYMENT`, `COMPASS_AUTH_ENABLED` and `COMPASS_AUTH_USERS`. Four feature flags gate whole routers — a module that is off never registers its routes: `COMPASS_PIPELINES`, `COMPASS_ESTIMATE`, `COMPASS_MISSIONS` and the newest, which switches on business functions. That module also reads a strict-catalog flag deciding whether a malformed manifest is skipped with a warning or refuses the start-up, and its notices use the same `COMPASS_SMTP_*` settings Routines already defines — with every unsent notice held and shown rather than lost.

Where to start reading

Follow one turn end to end: `frontend/src/app/home-chat/home-chat.ts` → `compass-api.service.ts` → `compass/home/routes.py` → `compass/home/engine.py` → `compass/common/gateway/azure_client.py`. That path touches the streaming protocol, the tool loop, the model gateway and the transcript store — which is most of what the other five modules also do.

Known limits in Phase 1

- Pipeline connection secrets are still a local file; the Key Vault backend is written and needs a vault URL.
- Design's generated artwork has not been exercised end to end — no stored design yet carries a drawn picture, and the sandboxed canvas may not load one when it does.
- Ownership fails open on legacy records, by design. New installs are unaffected; migrated ones need the `scripts/claim_unowned_*.py` pass.
- Business functions keeps one stand-in where a real system belongs: the directory — roles, names, addresses and who manages whom are a dict in code. The budgets and the policy thresholds used to sit beside it and no longer do: they are a versioned, admin-settable store, and the directory is the stand-in that is left.
- Its handlers are synchronous and its store is not, so a decision is written to memory and flushed by the route that caused it. A crash in the sub-millisecond window between the two loses that decision; the fix is an async feature contract, worth doing when that contract is next opened.

- The sign-in diagram stops at require_user, which is where identity ends and entitlement begins. It does not show the second gate Business functions adds after it — the breadth a feature's manifest grants each role, applied inside the handler — because that gate is per feature and a lane diagram has one lane for the API.
- The print-ready PDF at `docs/Compass-Architecture-Phase-1.pdf` in the repo is generated from an export of this document with reportlab. It was last built on 3 October 2026 and predates every Business functions section. Regenerate it after this document changes rather than editing the PDF.